

Projet SOC

Analyse de logs Linux et visualisation SIEM

Rapport d'analyse de logs Linux

Du traitement manuel a l'automatisation et a l'analyse SIEM

Auteur : KOMAN OHOUO Joseph

Formation : Reseaux et Securite Informatique

Outils : VS Code, Excel, Power Query, Python, Splunk Enterprise

Jeu de donnees : Linux_2k.log / Linuxlog.log

Rapport technique de projet cybersécurité

Analyse SOC junior

Table des matières

Résumé exécutif	4
1 Introduction	5
2 Contexte et objectifs du projet	6
2.1 Contexte	6
2.2 Objectifs	6
2.3 Outils utilisés	7
3 Présentation du jeu de données	8
3.1 Structure générale des logs	8
3.2 Décomposition d'une ligne de log	8
4 Méthodologie d'analyse	10
4.1 Vue d'ensemble de la démarche	10
4.2 Analyse manuelle	10
4.3 Structuration avec Excel et Power Query	11
4.4 Automatisation avec Python	13
4.5 Analyse SIEM avec Splunk	15
5 Critères de détection et de sévérité	17
5.1 Types d'événements étudiés	17

5.2	Critères de sévérité	17
6	Résultats de l'analyse	19
6.1	Résultats globaux	19
6.2	Cohérence des résultats entre Excel, Python et Splunk	19
6.3	Sources les plus actives	20
6.4	Événements critiques	21
6.5	Interprétation SOC	22
7	Automatisation de l'analyse avec Python	23
7.1	Objectif du script	23
7.2	Extrait du script	23
7.3	Résultats du script	25
8	Analyse SIEM avec Splunk	26
8.1	Principe de l'analyse SPL	26
8.2	Rôle de <code>_raw</code> et de <code>rex</code>	26
8.3	Recherche de validation dans Splunk	27
8.4	Requête SPL pour compter les échecs par source	27
8.5	Requête SPL avec classification de sévérité	28
8.6	Point technique : importance du découpage des événements	28
9	Recommandations de sécurité	29
9.1	Désactiver la connexion SSH directe au compte root	29
9.2	Mettre en place Fail2ban	29
9.3	Renforcer l'authentification SSH	29
9.4	Surveiller les sources répétitives	30
9.5	Mettre en place des alertes SIEM	30

10 Limites et perspectives d'évolution	31
10.1 Limites observées	31
10.2 Perspectives d'amélioration	31
10.3 Dashboard Splunk envisagé	32
11 Conclusion	33

Résumé exécutif

Ce rapport présente une analyse de logs Linux réalisée dans une démarche SOC. L'objectif principal est d'identifier des événements suspects liés aux authentifications SSH, aux utilisateurs inconnus et aux anomalies système à partir du fichier `Linux_2k.log`. Une version réimportée dans Splunk sous le nom `Linuxlog.log` a également été utilisée pour valider les résultats avec un découpage correct des événements.

La démarche suivie repose sur plusieurs niveaux de traitement. Une première analyse manuelle a permis de comprendre la structure des logs et d'identifier les informations essentielles : date, hôte, service, adresse source et utilisateur ciblé. Les données ont ensuite été structurées avec Excel et Power Query afin de faciliter les filtres, les regroupements et la classification des événements. Un script Python a été développé pour automatiser la détection des événements suspects. Enfin, les logs ont été ingérés dans Splunk afin d'expérimenter une approche SIEM avec des recherches SPL.

L'analyse met en évidence un volume important d'échecs d'authentification SSH, principalement contre le compte `root`. Plusieurs sources distantes apparaissent de manière répétée, ce qui suggère des comportements compatibles avec des tentatives de brute force SSH ou de reconnaissance de comptes. Les résultats obtenus avec Excel, Python et Splunk sont cohérents sur le nombre d'échecs d'authentification SSH : 490 événements.

Principaux résultats obtenus :

- 490 échecs d'authentification SSH identifiés ;
- 117 événements de type utilisateur inconnu ;
- 607 entrées suspectes détectées au total avec Python ;
- plusieurs sources distantes répétitives ciblant le compte `root` ;
- validation des 490 échecs d'authentification SSH dans Splunk après réimport propre du fichier.

1 Introduction

Les logs constituent une source essentielle d'information en cybersécurité. Ils permettent de comprendre ce qui s'est produit sur un système, d'identifier des comportements anormaux et de reconstruire une chronologie d'événements. Dans un contexte SOC, l'analyse de logs fait partie des compétences fondamentales d'un analyste sécurité.

Un serveur Linux enregistre de nombreux événements : connexions réussies, échecs d'authentification, changements de session, alertes système, erreurs de services ou encore tentatives d'accès à distance. Ces traces peuvent révéler des activités malveillantes, comme une attaque par force brute SSH, un scan de comptes utilisateurs ou une tentative d'accès au compte administrateur.

Ce projet vise à analyser un échantillon de logs Linux afin de détecter des événements suspects et d'en tirer des conclusions exploitables. L'approche adoptée est progressive : commencer par comprendre les logs à la main, structurer les données avec Excel, automatiser l'analyse avec Python, puis exploiter Splunk pour reproduire une démarche plus proche d'un environnement SOC.

2 Contexte et objectifs du projet

2.1 Contexte

Le fichier étudié est `Linux_2k.log`, un jeu de logs Linux contenant environ 2000 lignes. Ces logs sont représentatifs d'événements système Linux et contiennent notamment des traces d'authentification, des événements SSH, des ouvertures et fermetures de session, ainsi que des alertes système.

Dans ce projet, l'attention est portée principalement sur les événements liés à l'authentification, car ils constituent un indicateur important pour détecter des tentatives d'intrusion.

2.2 Objectifs

Les objectifs du projet sont les suivants :

- comprendre la structure d'une ligne de log Linux ;
- identifier manuellement les événements suspects ;
- structurer les logs dans Excel avec Power Query ;
- extraire les champs importants : IP source, utilisateur ciblé, type d'événement ;
- compter les occurrences par source ;
- classer les événements selon un niveau de sévérité ;
- automatiser l'analyse avec un script Python ;
- ingérer les logs dans Splunk et effectuer des recherches SPL ;
- comparer les résultats entre Excel, Python et Splunk ;
- proposer des recommandations de sécurité.

2.3 Outils utilisés

Outil	Rôle dans le projet
VS Code	Lecture du fichier de logs et développement du script Python.
Excel	Structuration des logs, filtrage, analyse manuelle et tableau croisé dynamique.
Power Query	Nettoyage et transformation des logs avant chargement dans Excel.
Python	Automatisation de la détection des événements suspects et export CSV.
Splunk Enterprise	Ingestion des logs, recherches SPL, extraction de champs et analyse SIEM.

TABLE 2.1 – Outils utilisés dans le projet

3 Présentation du jeu de données

3.1 Structure générale des logs

Une ligne de log Linux contient généralement plusieurs informations : la date, l'heure, le nom de l'hôte, le service concerné et le message de l'événement.

Exemple de ligne de log étudiée :

```
Jun 15 02:04:59 combo sshd(pam_unix)[20897]: authentication failure; logname= uid
=0 euid=0 tty=NODEVssh ruser= rhost=220-135-151-1.hinet-ip.hinet.net user=
root
```

Cette ligne indique une tentative d'authentification échouée sur le service SSH. L'adresse source est indiquée par le champ `rhost` et l'utilisateur ciblé par le champ `user`.

3.2 Décomposition d'une ligne de log

Élément	Signification
Jun 15 02:04:59	Date et heure de l'événement.
combo	Nom de l'hôte ayant généré l'événement.
sshd(pam_unix)	Service concerné, ici SSH avec PAM Unix.
authentication failure	Type d'événement : échec d'authentification.
rhost=220-135-151-1.hinet-ip.hinet.net	Source distante de la tentative.
user=root	Utilisateur ciblé par la tentative.

TABLE 3.1 – Décomposition d'une ligne de log Linux

Point important : dans une analyse SOC, les champs **rhost**, **user** et le type d'événement sont essentiels pour identifier qui tente d'accéder au système, quel compte est ciblé et quelle action a été observée.

4 Méthodologie d'analyse

4.1 Vue d'ensemble de la démarche

La démarche suivie repose sur quatre niveaux :

1. analyse manuelle des logs pour comprendre la structure ;
2. structuration des données avec Excel et Power Query ;
3. automatisation avec Python ;
4. exploitation dans Splunk avec des requêtes SPL.

Cette progression permet de passer d'une lecture brute des logs à une analyse plus industrialisée, proche d'un contexte SOC.

4.2 Analyse manuelle

La première étape a consisté à ouvrir le fichier `Linux_2k.log` dans VS Code afin d'observer les premières lignes et d'identifier les événements suspects.

Les principaux mots-clés recherchés sont :

- `authentication failure` : échec d'authentification ;
- `Failed password` : mot de passe SSH incorrect ;
- `user unknown` : utilisateur inconnu ;
- `invalid user` : utilisateur invalide ;
- `ALERT` : alerte système.

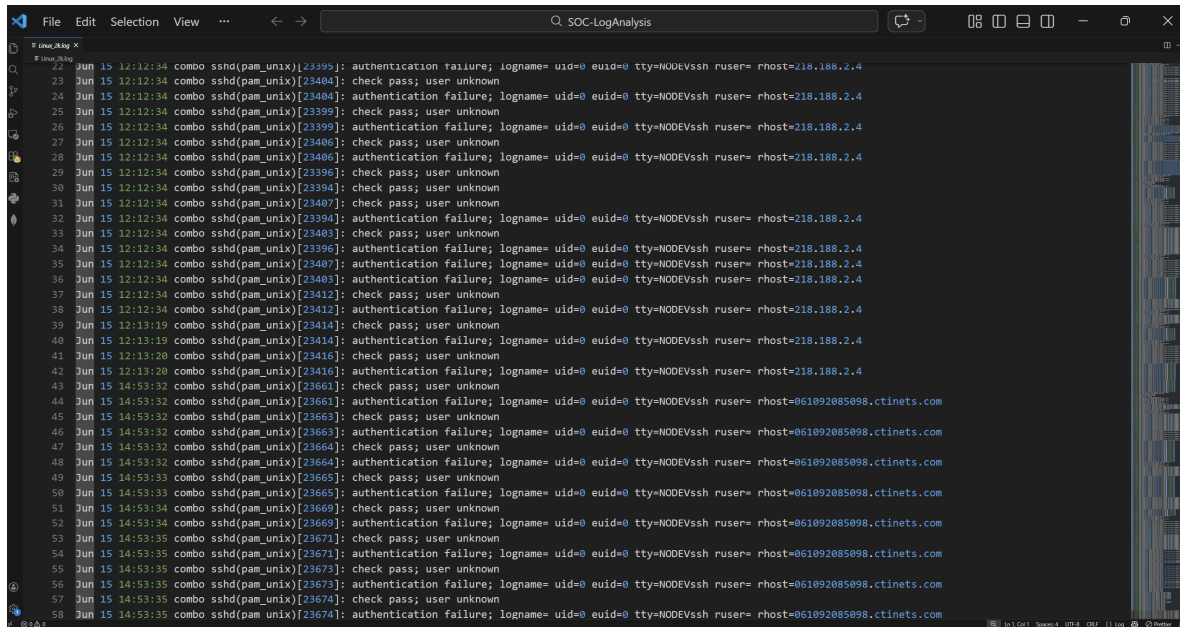


FIGURE 4.1 – Ouverture du fichier de logs dans VS Code

4.3 Structuration avec Excel et Power Query

Après l'analyse manuelle, les logs ont été importés dans Excel. L'objectif était de transformer un fichier texte brut en tableau exploitable.

Les colonnes retenues sont les suivantes :

- Date / Heure ;
- Hôte ;
- Message ;
- IP Source ;
- Utilisateur ciblé ;
- Type d'événement ;
- Nombre d'occurrences ;
- Sévérité.

Power Query a été utilisé pour nettoyer les données, fusionner les colonnes de date et extraire les champs utiles.

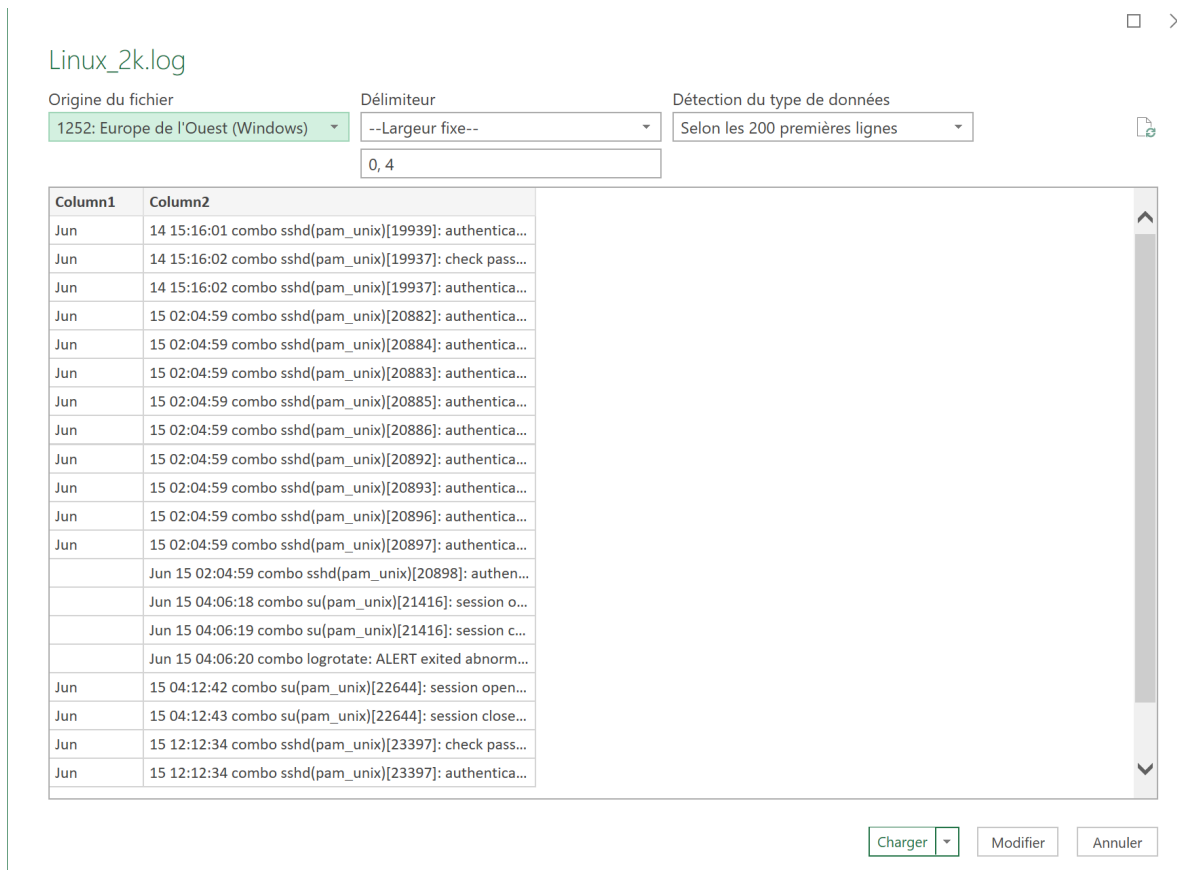


FIGURE 4.2 – Import du fichier de logs dans Excel avec Power Query

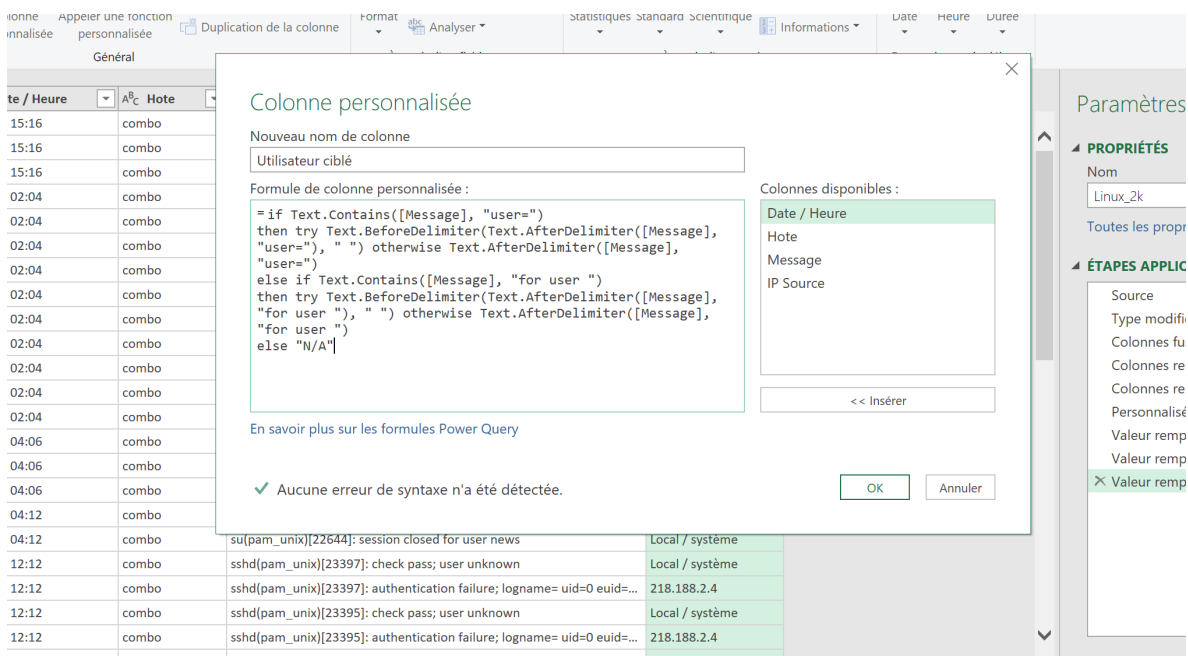


FIGURE 4.3 – Création de colonnes personnalisées dans Power Query

	A	B	C	D	E	F	G	H
	Date / Heure	Hôte	Message	IP Source	Utilisateur ciblé	Type d'événement	Nombre d'occurrence	Sévérité
2	Jun 14 15:16	combo	sshd(pam_uni		N/A	Échec d'authentification SSH	2	Faible
3	Jun 14 15:16	combo	sshd(pam_uni Local / système		unknown	Utilisateur inconnu	117	Moyenne
4	Jun 14 15:16	combo	sshd(pam_uni 218.188.2.4		N/A	Échec d'authentification SSH	13	Faible
5	Jun 15 02:04	combo	sshd(pam_uni 220.135.151.1		root	Échec d'authentification SSH	10	Élevée
6	Jun 15 02:04	combo	sshd(pam_uni 220.135.151.1		root	Échec d'authentification SSH	10	Élevée
7	Jun 15 02:04	combo	sshd(pam_uni 220.135.151.1		root	Échec d'authentification SSH	10	Élevée
8	Jun 15 02:04	combo	sshd(pam_uni 220.135.151.1		root	Échec d'authentification SSH	10	Élevée
9	Jun 15 02:04	combo	sshd(pam_uni 220.135.151.1		root	Échec d'authentification SSH	10	Élevée
10	Jun 15 02:04	combo	sshd(pam_uni 220.135.151.1		root	Échec d'authentification SSH	10	Élevée
11	Jun 15 02:04	combo	sshd(pam_uni 220.135.151.1		root	Échec d'authentification SSH	10	Élevée
12	Jun 15 02:04	combo	sshd(pam_uni 220.135.151.1		root	Échec d'authentification SSH	10	Élevée
13	Jun 15 02:04	combo	sshd(pam_uni 220.135.151.1		root	Échec d'authentification SSH	10	Élevée
14	Jun 15 02:04	combo	sshd(pam_uni 220.135.151.1		root	Échec d'authentification SSH	10	Élevée
15	Jun 15 04:06	combo	su(pam_unix) Local / système		cyrus	Ouverture de session	43	Faible
16	Jun 15 04:06	combo	su(pam_unix) Local / système		cyrus	Fermeture de session	43	Faible
17	Jun 15 04:06	combo	logrotate: ALI Local / système		N/A	Alerte système	43	Moyenne
18	Jun 15 04:12	combo	su(pam_unix) Local / système		news	Ouverture de session	43	Faible
19	Jun 15 04:12	combo	su(pam_unix) Local / système		news	Fermeture de session	43	Faible
20	Jun 15 12:12	combo	sshd(pam_uni Local / système		unknown	Utilisateur inconnu	117	Moyenne
21	Jun 15 12:12	combo	sshd(pam_uni 218.188.2.4		N/A	Échec d'authentification SSH	13	Faible

FIGURE 4.4 – Tableau final structuré dans Excel

4.4 Automatisation avec Python

Une fois les patterns identifiés manuellement et les données structurées dans Excel, un script Python a été développé afin d'automatiser l'analyse.

Le script effectue les actions suivantes :

- lecture du fichier `Linux_2k.log` ;
- détection des événements suspects ;
- extraction de l'adresse source ;
- extraction de l'utilisateur ciblé ;
- comptage des types d'événements ;
- export des résultats dans un fichier CSV.

```

2 # Script d'analyse automatique de logs Linux
3
4 import csv
5 import re
6
7 # Étape 1 : Lire le fichier de logs
8 with open("linux_2k.log", "r", encoding="utf-8", errors="ignore") as fichier:
9     logs = fichier.readlines()
10
11 # Étape 2 : Sélectionner les lignes 200 à 500
12 # Attention : en Python, l'index commence à 0
13 # logs[199:500] = lignes 200 à 500
14 sous_ensemble = logs[199:500]
15
16 # Étape 3 : Préparer la liste et les compteurs
17 entrees_suspectes = []
18
19 compteur_echec_connexion = 0
20 compteur_echec_auth = 0
21 compteur_utilisateur_inconnu = 0
22
23 # Étape 4 : Fonction pour extraire l'IP source
24 def extraire_ip(ligne):
25     match = re.search(r"rhost=([^\ ]+)", ligne)
26     if match:
27         ip = match.group(1)
28
29     # Nettoyage de certains noms DNS

```

FIGURE 4.5 – Script Python d'analyse automatique des logs

```

PS C:\Users\koman\Desktop\SOC-LogAnalysis> python .\analyse_logs.py
[Échec d'authentification SSH] Jun 28 21:42:46 | check pass; user unknown
[Échec d'authentification SSH] Jun 28 21:42:46 | IP: 211.115.206.155 | Utilisateur: N/A
[Échec d'authentification SSH] Jun 29 10:08:19 | IP: csnu.csnuok.edu | Utilisateur: root
[Échec d'authentification SSH] Jun 29 10:08:19 | authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=csnu.csnuok.edu user=root
[Échec d'authentification SSH] Jun 29 10:08:19 | IP: csnu.csnuok.edu | Utilisateur: root
[Échec d'authentification SSH] Jun 29 10:08:19 | authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=csnu.csnuok.edu user=root
[Échec d'authentification SSH] Jun 29 10:08:19 | IP: csnu.csnuok.edu | Utilisateur: root
[Échec d'authentification SSH] Jun 29 10:08:19 | authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=csnu.csnuok.edu user=root
[Échec d'authentification SSH] Jun 29 10:08:19 | IP: csnu.csnuok.edu | Utilisateur: root
[Échec d'authentification SSH] Jun 29 10:08:19 | authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=csnu.csnuok.edu user=root
[Échec d'authentification SSH] Jun 29 10:08:20 | IP: csnu.csnuok.edu | Utilisateur: root
[Échec d'authentification SSH] Jun 29 10:08:20 | authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=csnu.csnuok.edu user=root
[Échec d'authentification SSH] Jun 29 10:08:20 | IP: csnu.csnuok.edu | Utilisateur: root
[Échec d'authentification SSH] Jun 29 10:08:20 | authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=csnu.csnuok.edu user=root
[Échec d'authentification SSH] Jun 29 10:08:20 | IP: csnu.csnuok.edu | Utilisateur: root
[Échec d'authentification SSH] Jun 29 10:08:20 | authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=csnu.csnuok.edu user=root
[Échec d'authentification SSH] Jun 29 12:11:53 | IP: h64.187.1.131.gtconnect.net | Utilisateur: root
[Échec d'authentification SSH] Jun 29 12:11:55 | IP: h64.187.1.131.gtconnect.net | Utilisateur: root
[Échec d'authentification SSH] Jun 29 12:11:55 | authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=h64-187-1-131.gtconnect.net user=root
[Échec d'authentification SSH] Jun 29 12:11:57 | IP: h64.187.1.131.gtconnect.net | Utilisateur: root
[Échec d'authentification SSH] Jun 29 12:11:57 | authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=h64-187-1-131.gtconnect.net user=root
[Échec d'authentification SSH] Jun 29 12:11:59 | IP: h64.187.1.131.gtconnect.net | Utilisateur: root
[Échec d'authentification SSH] Jun 29 12:11:59 | authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=h64-187-1-131.gtconnect.net user=root
[Échec d'authentification SSH] Jun 29 12:12:01 | IP: h64.187.1.131.gtconnect.net | Utilisateur: root
[Échec d'authentification SSH] Jun 29 12:12:01 | authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=h64-187-1-131.gtconnect.net user=root
[Échec d'authentification SSH] Jun 29 12:12:02 | IP: h64.187.1.131.gtconnect.net | Utilisateur: root
[Échec d'authentification SSH] Jun 29 12:12:02 | authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=h64-187-1-131.gtconnect.net user=root
[Échec d'authentification SSH] Jun 29 12:12:03 | IP: h64.187.1.131.gtconnect.net | Utilisateur: root
[Échec d'authentification SSH] Jun 29 12:12:03 | authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=h64-187-1-131.gtconnect.net user=root
[Échec d'authentification SSH] Jun 29 12:12:05 | IP: h64.187.1.131.gtconnect.net | Utilisateur: root
[Échec d'authentification SSH] Jun 29 12:12:05 | authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=h64-187-1-131.gtconnect.net user=root
[Échec d'authentification SSH] Jun 29 12:12:06 | IP: h64.187.1.131.gtconnect.net | Utilisateur: root
[Échec d'authentification SSH] Jun 29 12:12:06 | authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=h64-187-1-131.gtconnect.net user=root
[Échec d'authentification SSH] Jun 29 12:12:10 | IP: h64.187.1.131.gtconnect.net | Utilisateur: root
[Échec d'authentification SSH] Jun 29 12:12:10 | authentication failure; logname= uid=0 euid=0 tty=NODEVssh ruser= rhost=h64-187-1-131.gtconnect.net user=root

--- Résumé de l'analyse ---
Échecs de connexion SSH : 0
Échecs d'authentification SSH : 114
Utilisateurs inconnus : 26
Total : 140 entrées suspectes

Résultats sauvegardés dans logs_suspects.csv
% PS C:\Users\koman\Desktop\SOC-LogAnalysis>

```

FIGURE 4.6 – Exécution du script Python et affichage des résultats

4.5 Analyse SIEM avec Splunk

La dernière étape a consisté à ingérer le fichier de logs dans Splunk Enterprise afin de réaliser des recherches SPL. Après un réimport propre du fichier sous le nom `Linuxlog.log`, Splunk a retourné 490 événements pour la recherche `authentication failure`, ce qui confirme les résultats obtenus avec Excel et Python.

Splunk permet de rechercher rapidement dans les logs, d'extraire des champs, de compter les occurrences et de produire des visualisations. Dans ce projet, les recherches SPL ont été utilisées pour identifier les sources générant des échecs d'authentification SSH.

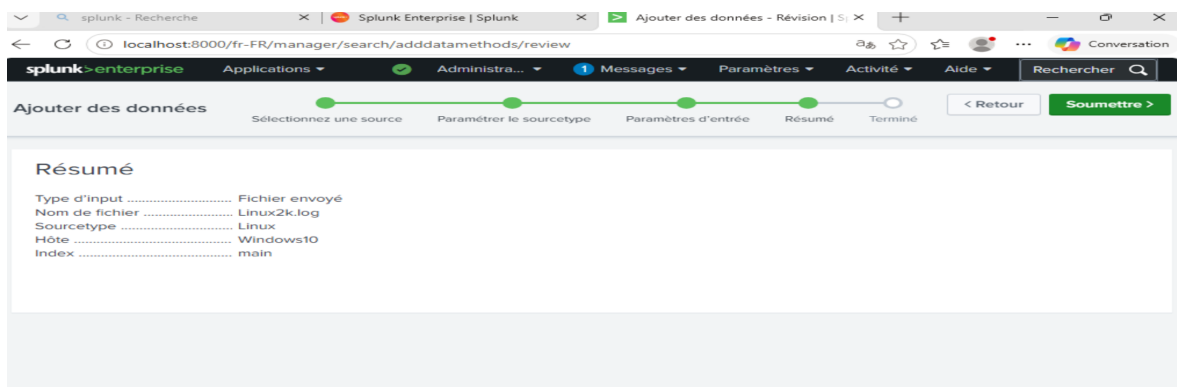


FIGURE 4.7 – Import du fichier de logs dans Splunk

The screenshot shows the Splunk Enterprise search interface. The search bar contains the query: `source="Linuxlog.log" sourcetype="linux2" authentication failure`. The results show 490 events. The interface includes a sidebar with navigation options like 'Recherche', 'Analytique', 'Jeux de données', 'Rapports', 'Alertes', 'Tableaux de bord', and 'Modules'. The main content area displays the search results in a table format. The table has columns for 'i', 'Heure', and 'Événement'. The first two rows of the table are visible, showing authentication failure events from July 26, 2026, at 08:56:36.000. The events include details like 'uid=0', 'euid=0', 'tty=NODEVssh', 'ruser=', 'rhost=207.243.167.114', 'user=root', and 'host = Windows10'. The search results are displayed in a table format with columns for 'i', 'Heure', and 'Événement'. The first two rows of the table are visible, showing authentication failure events from July 26, 2026, at 08:56:36.000. The events include details like 'uid=0', 'euid=0', 'tty=NODEVssh', 'ruser=', 'rhost=207.243.167.114', 'user=root', and 'host = Windows10'.

i	Heure	Événement
>	21/06/2026 08:56:36.000	Jul 26 07:04:12 combo sshd(pam_unix)[28886]: authentication failure; logname=uid=0 euid=0 tty=NODEVssh ruser= rhost=207.243.167.114 user=root host = Windows10 source = Linuxlog.log sourcetype = linux2
>	21/06/2026 08:56:36.000	Jul 26 07:04:07 combo sshd(pam_unix)[28884]: authentication failure; logname=uid=0 euid=0 tty=NODEVssh ruser= rhost=207.243.167.114 user=root host = Windows10 source = Linuxlog.log sourcetype = linux2

FIGURE 4.8 – Recherche Splunk retournant 490 échecs d'authentification SSH

5 Critères de détection et de sévérité

5.1 Types d'événements étudiés

Les événements suspects ont été classés en plusieurs catégories.

Type d'événement	Interprétation SOC
Échec d'authentification SSH	Tentative d'accès échouée sur le service SSH. Peut être bénin si isolé, mais suspect si répété.
Utilisateur inconnu	Tentative d'accès avec un compte inexistant. Peut indiquer une phase de reconnaissance.
Utilisateur invalide	Tentative d'accès avec un nom d'utilisateur non valide.
Ouverture de session	Connexion ou changement de session local. À surveiller selon le contexte.
Fermeture de session	Fin de session utilisateur.
Alerte système	Anomalie système nécessitant une vérification.

TABLE 5.1 – Types d'événements identifiés

5.2 Critères de sévérité

La sévérité a été déterminée en fonction du compte ciblé, de la répétition des événements et du type de message observé.

Critère	Sévérité	Justification
Compte <code>root</code> ciblé avec une IP répétée et un volume élevé	Élevée	Tentative probable de brute force contre un compte administrateur.
Événements <code>user unknown</code> ou <code>invalid user</code>	Moyenne	Comportement compatible avec une phase de reconnaissance ou de scan de comptes.
Échec isolé sur un compte non privilégié	Faible	Peut correspondre à une erreur utilisateur ou un événement isolé.
Événement <code>ALERT exited abnormally</code>	Moyenne	Peut indiquer une instabilité système ou une anomalie à investiguer.

TABLE 5.2 – Critères de classification de la sévérité

6 Résultats de l'analyse

6.1 Résultats globaux

L'analyse automatisée avec Python a permis d'obtenir les résultats suivants :

- Échecs de connexion SSH : 0 ;
- Échecs d'authentification SSH : 490 ;
- Utilisateurs inconnus : 117 ;
- Total des entrées suspectes : 607.

Ces résultats montrent que les échecs d'authentification SSH constituent la majorité des événements suspects observés.

6.2 Cohérence des résultats entre Excel, Python et Splunk

Le résultat le plus important du projet est la cohérence obtenue entre les trois méthodes d'analyse. Excel, Python et Splunk identifient le même nombre d'échecs d'authentification SSH.

Méthode	Résultat	Commentaire
Excel / Power Query	490	Comptage après structuration et filtrage des logs.
Python	490	Détection automatique des lignes contenant <code>authentication failure</code> .
Splunk	490	Recherche SPL sur <code>source="Linuxlog.log" sourcetype="linux2" authentication failure</code> .

TABLE 6.1 – Validation croisée des échecs d'authentification SSH

Cette cohérence renforce la fiabilité de l'analyse, car elle montre que les résultats ne dé-

pendent pas d'un seul outil.

6.3 Sources les plus actives

L'analyse par tableau croisé dynamique dans Excel a permis d'identifier les sources les plus actives. La catégorie `Local / système` regroupe les événements qui ne disposent pas d'adresse source distante exploitable. Elle n'est donc pas considérée comme une source externe d'attaque.

IP ou nom d'hôte source	Occurrences	Interprétation
150.183.249.110	80	Source distante fortement répétitive.
n219076184117.netvigator.com	23	Tentatives répétées contre <code>root</code> .
207.243.167.114	23	Volume élevé d'échecs SSH.
60.30.224.116	20	Activité suspecte répétée.
195.129.24.210	15	Tentatives multiples observées.
218.188.2.4	13	Échecs répétés sur SSH.

TABLE 6.2 – Sources distantes les plus actives

	A	B	C	D
3	IP source	Nombre d'occurrence		
4	Local / système	1510		
5	150.183.249.110	80		
6	n219076184117.netvigator.com	23		
7	207.243.167.114	23		
8	60.30.224.116	20		
9	195.129.24.210	15		
10	h64-187-1-131.gtconnect.net	13		
11	218.188.2.4	13		
12	220.117.241.87	13		
13	211.9.58.217	10		
14	c51471f2c.cable.wanadoo.nl	10		
15	82.77.200.128	10		
16	202.181.236.180	10		
17	211.214.161.141	10		
18	zummit.com	10		
19	211-76-104-65.ebix.net.tw	10		
20	209.152.168.249	10		
21	adsl-70-242-75-179.dsl.ksc2mo.swbell.net	10		
22	p15105218.pureserver.info	10		
23	cscu.psu.edu	10		

FIGURE 6.1 – Tableau croisé dynamique des occurrences par source

6.4 Événements critiques

Les événements classés en sévérité élevée correspondent principalement à des échecs d'authentification SSH répétés contre le compte `root`. Ce comportement est particulièrement sensible, car le compte `root` dispose des privilèges les plus élevés sur un système Linux.

	A	B	C	D	E	F	G	H
	Date / Heure	Hôte	Message	IP Source	Utilisateur ciblé	Type d'événement	Nombre d'occurrence	Sévérité
5	Jun 15 02:04	combo	sshd(pam_uni 220.135.151.1		root	Échec d'authentification SSH	10	Élevée
6	Jun 15 02:04	combo	sshd(pam_uni 220.135.151.1		root	Échec d'authentification SSH	10	Élevée
7	Jun 15 02:04	combo	sshd(pam_uni 220.135.151.1		root	Échec d'authentification SSH	10	Élevée
8	Jun 15 02:04	combo	sshd(pam_uni 220.135.151.1		root	Échec d'authentification SSH	10	Élevée
9	Jun 15 02:04	combo	sshd(pam_uni 220.135.151.1		root	Échec d'authentification SSH	10	Élevée
10	Jun 15 02:04	combo	sshd(pam_uni 220.135.151.1		root	Échec d'authentification SSH	10	Élevée
11	Jun 15 02:04	combo	sshd(pam_uni 220.135.151.1		root	Échec d'authentification SSH	10	Élevée
12	Jun 15 02:04	combo	sshd(pam_uni 220.135.151.1		root	Échec d'authentification SSH	10	Élevée
13	Jun 15 02:04	combo	sshd(pam_uni 220.135.151.1		root	Échec d'authentification SSH	10	Élevée
14	Jun 15 02:04	combo	sshd(pam_uni 220.135.151.1		root	Échec d'authentification SSH	10	Élevée
200	Jun 22 03:17	combo	sshd(pam_uni n219076184117.netvigator.com		root	Échec d'authentification SSH	23	Élevée
201	Jun 22 03:17	combo	sshd(pam_uni n219076184117.netvigator.com		root	Échec d'authentification SSH	23	Élevée
202	Jun 22 03:17	combo	sshd(pam_uni n219076184117.netvigator.com		root	Échec d'authentification SSH	23	Élevée
203	Jun 22 03:17	combo	sshd(pam_uni n219076184117.netvigator.com		root	Échec d'authentification SSH	23	Élevée
204	Jun 22 03:17	combo	sshd(pam_uni n219076184117.netvigator.com		root	Échec d'authentification SSH	23	Élevée
205	Jun 22 03:17	combo	sshd(pam_uni n219076184117.netvigator.com		root	Échec d'authentification SSH	23	Élevée
206	Jun 22 03:17	combo	sshd(pam_uni n219076184117.netvigator.com		root	Échec d'authentification SSH	23	Élevée
207	Jun 22 03:17	combo	sshd(pam_uni n219076184117.netvigator.com		root	Échec d'authentification SSH	23	Élevée
208	Jun 22 03:17	combo	sshd(pam_uni n219076184117.netvigator.com		root	Échec d'authentification SSH	23	Élevée
209	Jun 22 03:17	combo	sshd(pam_uni n219076184117.netvigator.com		root	Échec d'authentification SSH	23	Élevée

FIGURE 6.2 – Filtrage des événements de sévérité élevée dans Excel

6.5 Interprétation SOC

Les résultats obtenus suggèrent plusieurs comportements suspects :

- des tentatives répétées d'authentification SSH depuis les mêmes sources ;
- un ciblage fréquent du compte `root` ;
- des tests de comptes inexistantes avec `user unknown` ;
- des anomalies système nécessitant une vérification.

Ces éléments sont compatibles avec des activités de reconnaissance et des tentatives de brute force SSH. Dans un environnement réel, ces événements devraient être corrélés avec les journaux de connexion réussie afin de vérifier si une tentative échouée a été suivie d'un accès valide.

7 Automatisation de l'analyse avec Python

7.1 Objectif du script

Le script Python a été développé pour automatiser la détection des événements suspects et éviter une analyse manuelle ligne par ligne.

L'objectif n'est pas seulement de compter des chaînes de caractères, mais de transformer des logs bruts en informations exploitables : type d'événement, IP source, utilisateur ciblé et ligne de log associée.

7.2 Extrait du script

```
import csv
import re

with open("Linux_2k.log", "r", encoding="utf-8", errors="ignore") as fichier:
    logs = fichier.readlines()

entrees_suspectes = []
compteur_echec_connexion = 0
compteur_echec_auth = 0
compteur_utilisateur_inconnu = 0

def extraire_ip(ligne):
    match = re.search(r"rhost=([^\ ]+)", ligne)
    if match:
        return match.group(1)
    return "Local / systeme"
```



```
def extraire_utilisateur(ligne):
    match_user = re.search(r"\suser=([^\s]+)", ligne)
    if match_user:
        return match_user.group(1)

    match_for_user = re.search(r"for user ([^\s]+)", ligne)
    if match_for_user:
        return match_for_user.group(1)

    if "user unknown" in ligne:
        return "unknown"

    return "N/A"

for ligne in logs:
    ligne = ligne.strip()

    if "Failed password" in ligne:
        type_evt = "Echec connexion SSH"
        compteur_echec_connexion += 1

    elif "authentication failure" in ligne:
        type_evt = "Echec authentication SSH"
        compteur_echec_auth += 1

    elif "user unknown" in ligne or "invalid user" in ligne:
        type_evt = "Utilisateur inconnu"
        compteur_utilisateur_inconnu += 1

    else:
        continue

    ip_source = extraire_ip(ligne)
    utilisateur = extraire_utilisateur(ligne)

    entrees_suspectes.append([
        ip_source,
        utilisateur,
        type_evt,
        ligne
    ])
])
```

```

print("=== Resume de l'analyse ===")
print(f"Echecs de connexion SSH : {compteur_echec_connexion}")
print(f"Echecs d'authentification SSH : {compteur_echec_auth}")
print(f"Utilisateurs inconnus : {compteur_utilisateur_inconnu}")
print(f"Total : {len(entrees_suspectes)} entrees suspectes")

with open("logs_suspects.csv", "w", newline="", encoding="utf-8") as fichier_csv:
    writer = csv.writer(fichier_csv)
    writer.writerow(["IP Source", "Utilisateur cible", "Type d'événement", "Ligne de log"])
    writer.writerows(entrees_suspectes)

```

7.3 Résultats du script

Le script a confirmé les résultats obtenus avec Excel. Il a détecté un volume important d'événements suspects, notamment 490 échecs d'authentification SSH et 117 événements liés à des utilisateurs inconnus.

logs_suspects.csv

Origine du fichier: 1252: Europe de l'Ouest (Windows) | Délimiteur: Virgule | Détection du type de données: Selon les 200 premières lignes

Column1	Column2	Column3	Column4	Column5
Date / Heure	IP Source	Utilisateur ciblé	Type d'événement	Ligne de log
Jun 22 03:17:26	n219076184117.netvigator.com	root	Ã%chech dâ€™authentification SSH	Jun 22 03:17:26 combo sshd(pam_unix)[16206]: auther
Jun 22 03:17:35	n219076184117.netvigator.com	root	Ã%chech dâ€™authentification SSH	Jun 22 03:17:35 combo sshd(pam_unix)[16210]: auther
Jun 22 03:17:36	n219076184117.netvigator.com	root	Ã%chech dâ€™authentification SSH	Jun 22 03:17:36 combo sshd(pam_unix)[16212]: auther
Jun 22 03:17:36	n219076184117.netvigator.com	root	Ã%chech dâ€™authentification SSH	Jun 22 03:17:36 combo sshd(pam_unix)[16213]: auther
Jun 22 03:17:45	n219076184117.netvigator.com	root	Ã%chech dâ€™authentification SSH	Jun 22 03:17:45 combo sshd(pam_unix)[16216]: auther
Jun 22 03:17:46	n219076184117.netvigator.com	root	Ã%chech dâ€™authentification SSH	Jun 22 03:17:46 combo sshd(pam_unix)[16218]: auther
Jun 22 03:17:46	n219076184117.netvigator.com	root	Ã%chech dâ€™authentification SSH	Jun 22 03:17:46 combo sshd(pam_unix)[16219]: auther
Jun 22 03:17:52	n219076184117.netvigator.com	root	Ã%chech dâ€™authentification SSH	Jun 22 03:17:52 combo sshd(pam_unix)[16222]: auther
Jun 22 03:17:55	n219076184117.netvigator.com	root	Ã%chech dâ€™authentification SSH	Jun 22 03:17:55 combo sshd(pam_unix)[16224]: auther
Jun 22 03:17:56	n219076184117.netvigator.com	root	Ã%chech dâ€™authentification SSH	Jun 22 03:17:56 combo sshd(pam_unix)[16226]: auther
Jun 22 03:17:56	n219076184117.netvigator.com	root	Ã%chech dâ€™authentification SSH	Jun 22 03:17:56 combo sshd(pam_unix)[16227]: auther
Jun 22 03:18:02	n219076184117.netvigator.com	root	Ã%chech dâ€™authentification SSH	Jun 22 03:18:02 combo sshd(pam_unix)[16230]: auther
Jun 22 03:18:05	n219076184117.netvigator.com	root	Ã%chech dâ€™authentification SSH	Jun 22 03:18:05 combo sshd(pam_unix)[16232]: auther
Jun 22 03:18:06	n219076184117.netvigator.com	root	Ã%chech dâ€™authentification SSH	Jun 22 03:18:06 combo sshd(pam_unix)[16234]: auther
Jun 22 03:18:06	n219076184117.netvigator.com	root	Ã%chech dâ€™authentification SSH	Jun 22 03:18:06 combo sshd(pam_unix)[16235]: auther
Jun 22 03:18:10	n219076184117.netvigator.com	root	Ã%chech dâ€™authentification SSH	Jun 22 03:18:10 combo sshd(pam_unix)[16238]: auther
Jun 22 03:18:12	n219076184117.netvigator.com	root	Ã%chech dâ€™authentification SSH	Jun 22 03:18:12 combo sshd(pam_unix)[16240]: auther
Jun 22 03:18:15	n219076184117.netvigator.com	root	Ã%chech dâ€™authentification SSH	Jun 22 03:18:15 combo sshd(pam_unix)[16242]: auther
Jun 22 03:18:16	n219076184117.netvigator.com	root	Ã%chech dâ€™authentification SSH	Jun 22 03:18:16 combo sshd(pam_unix)[16244]: auther

Charger | Modifier | Annuler

FIGURE 7.1 – Ouverture du résultat CSV généré par le script dans Excel

8 Analyse SIEM avec Splunk

8.1 Principe de l'analyse SPL

Dans Splunk, les données sont analysées à l'aide du langage SPL. Une requête SPL permet de chercher les logs, d'extraire des champs, de filtrer les résultats, de compter les occurrences et d'afficher les résultats sous forme de tableau ou de visualisation.

Une requête suit généralement cette logique :

```
chercher les logs
| extraire les champs utiles
| filtrer les resultats
| compter / regrouper
| creer des colonnes d'analyse
| trier
| afficher
```

8.2 Rôle de `_raw` et de `rex`

Dans Splunk, le champ `_raw` contient l'événement brut tel qu'il a été indexé. C'est la source originale à partir de laquelle les champs peuvent être extraits.

La commande `rex` permet d'extraire une partie de `_raw` avec une expression régulière. Par exemple, pour extraire l'adresse source :

```
| rex field=_raw "rhost=(?<rhost>\S+)"
```

Pour extraire l'utilisateur ciblé :

```
| rex field=_raw "\suser=(?<user>\S+)"
```

8.3 Recherche de validation dans Splunk

Après réimportation propre du fichier de logs, la recherche suivante retourne 490 événements :

```
source="Linuxlog.log" sourcetype="linux2" authentication failure
```

Ce résultat est identique à celui obtenu avec Excel et Python. Cela confirme que les logs sont correctement exploitables dans Splunk et que la recherche SPL permet de retrouver rapidement les événements d'authentification échouée.

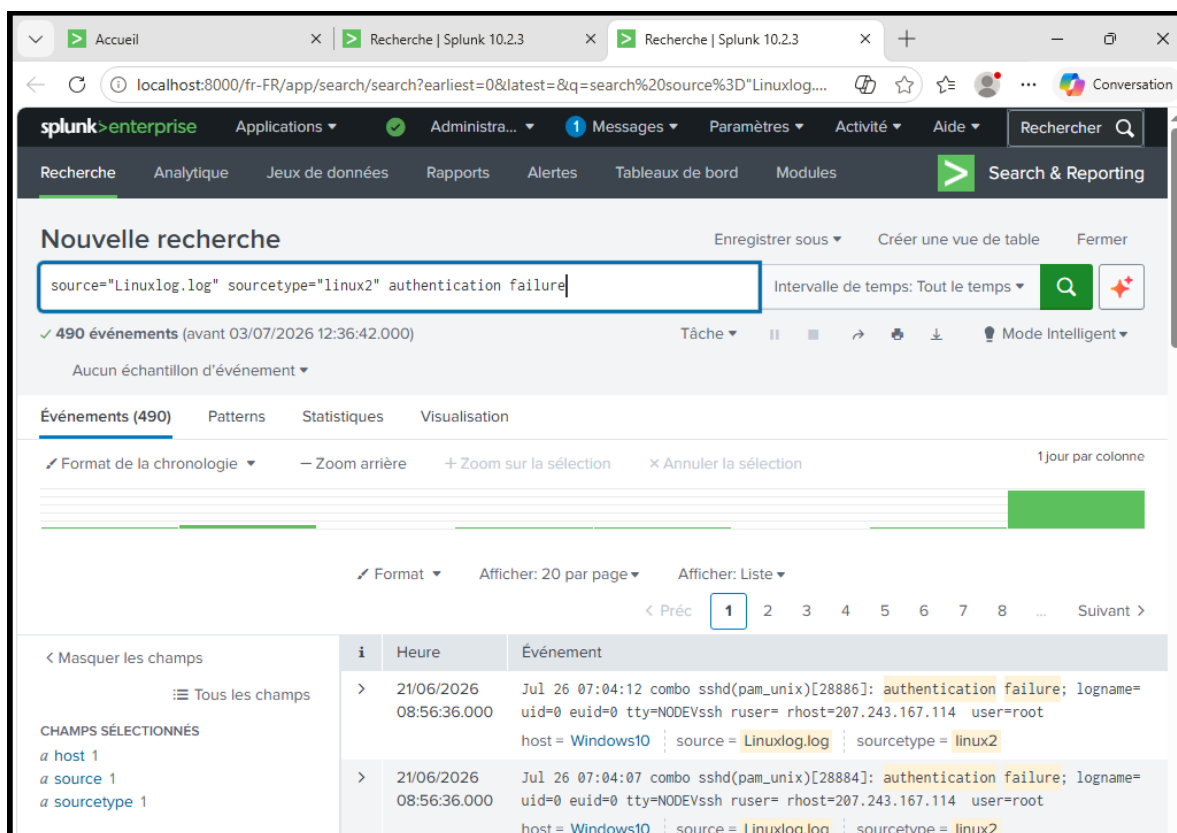


FIGURE 8.1 – Validation dans Splunk des 490 échecs d'authentification SSH

8.4 Requête SPL pour compter les échecs par source

```
source="Linuxlog.log" sourcetype="linux2" "authentication failure"
| rex field=_raw "rhost=(?<rhost>\S+)"
| rex field=_raw "\suser=(?<user>\S+)"
| stats count as occurrences by rhost user
| sort - occurrences
```

Cette requête permet d'identifier les sources qui génèrent le plus d'échecs d'authentification SSH, ainsi que les utilisateurs ciblés.

8.5 Requête SPL avec classification de sévérité

```
source="Linuxlog.log" sourcetype="linux2" "authentication failure"
| rex field=_raw "rhost=(?<rhost>\S+)"
| rex field=_raw "\suser=(?<user>\S+)"
| stats count as occurrences by rhost user
| eval severite=case(
    user="root" AND occurrences>=5, "Elevee",
    user="root" AND occurrences<5, "Moyenne",
    true(), "Faible"
)
| sort - occurrences
| table rhost user occurrences severite
```

8.6 Point technique : importance du découpage des événements

Un point important a été observé pendant l'utilisation de Splunk : le résultat dépend fortement de la manière dont le fichier est importé. Splunk ne compte pas directement les lignes d'un fichier ; il compte les événements indexés. Si plusieurs lignes sont regroupées dans un même événement, le résultat peut être différent de celui obtenu avec Excel ou Python.

Après réimportation propre du fichier `Linuxlog.log` avec le sourcetype `linux2`, la recherche sur `authentication failure` a retourné 490 événements, ce qui valide la cohérence avec Excel et Python.

Enseignement retenu : pour des logs Linux de type syslog, il est essentiel de vérifier que Splunk découpe correctement les données. Lorsque le découpage est correct, l'analyse SPL devient fiable, rapide et beaucoup plus efficace qu'une analyse manuelle.

9 Recommandations de sécurité

À partir des événements observés, plusieurs recommandations peuvent être formulées.

9.1 Désactiver la connexion SSH directe au compte `root`

Le compte `root` est une cible prioritaire pour les attaquants. La connexion directe au compte `root` via SSH doit être désactivée.

Exemple de configuration à vérifier dans `/etc/ssh/sshd_config` :

```
PermitRootLogin no
```

9.2 Mettre en place Fail2ban

Fail2ban permet de bloquer automatiquement une adresse IP après plusieurs échecs d'authentification. Cela permet de limiter les attaques par force brute.

9.3 Renforcer l'authentification SSH

Il est recommandé de :

- privilégier l'authentification par clé SSH ;
- désactiver l'authentification par mot de passe si possible ;
- utiliser des mots de passe robustes pour les comptes autorisés ;
- limiter les comptes autorisés à se connecter en SSH.

9.4 Surveiller les sources répétitives

Les adresses ou noms d'hôtes apparaissant de manière répétée doivent être surveillés, corrélés avec d'autres logs et éventuellement bloqués au niveau du pare-feu.

9.5 Mettre en place des alertes SIEM

Une alerte Splunk pourrait être créée pour détecter automatiquement un nombre élevé d'échecs SSH sur le compte `root` depuis une même source dans une fenêtre temporelle courte.

Exemple de logique d'alerte :

```
source="Linuxlog.log" sourcetype="linux2" "authentication failure"
| rex field=_raw "rhost=(?<rhost>\S+)"
| rex field=_raw "\suser=(?<user>\S+)"
| search user="root"
| stats count as occurrences by rhost user
| where occurrences>=5
```

10 Limites et perspectives d'évolution

10.1 Limites observées

Ce projet a permis de réaliser une analyse complète des logs avec plusieurs outils. Cependant, certaines limites ont été identifiées.

La première limite concerne le contexte disponible. Les logs étudiés permettent d'observer des tentatives d'authentification et des anomalies, mais ils ne fournissent pas à eux seuls une vision complète de l'environnement réseau. Pour confirmer une compromission, il faudrait corréler ces logs avec d'autres sources : pare-feu, IDS/IPS, logs applicatifs, logs système supplémentaires et journaux de connexions réussies.

La deuxième limite concerne la qualification des sources. Certaines valeurs apparaissent sous forme d'adresses IP, tandis que d'autres apparaissent sous forme de noms DNS. Une étape supplémentaire d'enrichissement pourrait permettre d'obtenir plus de contexte sur ces sources.

10.2 Perspectives d'amélioration

Plusieurs améliorations peuvent être réalisées dans une prochaine étape :

- créer un dashboard Splunk complet ;
- mettre en place des alertes SPL ;
- analyser d'autres sources de logs, comme Apache, Windows Event Logs ou pfSense ;
- corréler les événements SSH avec des logs réseau ;
- enrichir les IP suspectes avec des informations de réputation ou de threat intelligence ;
- automatiser davantage le calcul de la sévérité dans Splunk.

10.3 Dashboard Splunk envisagé

Un dashboard Splunk pourrait contenir les panneaux suivants :

Panneau	Objectif
Nombre total d'événements suspects	Mesurer le volume global d'activité suspecte.
Top 10 des IP sources	Identifier les sources les plus agressives.
Comptes les plus ciblés	Vérifier si <code>root</code> est principalement visé.
Échecs SSH par heure	Visualiser les pics d'activité.
Répartition par sévérité	Prioriser les investigations.

TABLE 10.1 – Dashboard Splunk envisagé

11 Conclusion

Ce projet a permis de réaliser une analyse progressive de logs Linux dans une logique SOC. L'analyse manuelle a d'abord permis de comprendre la structure des événements et d'identifier les champs importants. Excel et Power Query ont ensuite facilité la structuration des données, le filtrage et le comptage des occurrences. Python a permis d'automatiser la détection des événements suspects et de générer un fichier exploitable. Enfin, Splunk a permis d'expérimenter une approche SIEM avec des recherches SPL.

Les résultats montrent un volume important d'échecs d'authentification SSH et de tentatives sur des comptes inconnus. Le ciblage répété du compte `root` par certaines sources distantes constitue un indicateur fort d'activité suspecte, compatible avec des tentatives de brute force SSH.

La validation croisée entre Excel, Python et Splunk est un point fort du projet : les trois méthodes identifient 490 échecs d'authentification SSH. Cette cohérence montre que les résultats sont fiables et que la démarche d'analyse est solide.

Ce projet met aussi en évidence une notion essentielle en SIEM : la différence entre une ligne de log et un événement indexé. Comprendre cette différence est important pour interpréter correctement les résultats d'un outil comme Splunk.

Au final, cette démarche montre l'intérêt de combiner compréhension manuelle, structuration de données, automatisation Python et analyse SIEM pour produire une investigation plus fiable et exploitable.